

tf.compat.v1.set_random_seed Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/set_random_seed

Sets the graph-level random seed for the default graph.

Function Signatures

```
tf.compat.v1.set_random_seed( seed )
```

Code Examples

```
tf.compat.v1.set_random_seed(  
    seed  
)
```

```
tf.compat.v1.set_random_seed(  
    seed  
)
```

```
a = tf.random.uniform([1])
b = tf.random.normal([1])
print("Session 1")
with tf.compat.v1.Session() as sess1:
    print(sess1.run(a)) # generates 'A1'
    print(sess1.run(a)) # generates 'A2'
    print(sess1.run(b)) # generates 'B1'
    print(sess1.run(b)) # generates 'B2'
print("Session 2")
with tf.compat.v1.Session() as sess2:
    print(sess2.run(a)) # generates 'A3'
    print(sess2.run(a)) # generates 'A4'
    print(sess2.run(b)) # generates 'B3'
    print(sess2.run(b)) # generates 'B4'
```

```
a = tf.random.uniform([1])
b = tf.random.normal([1])
print("Session 1")
with tf.compat.v1.Session() as sess1:
    print(sess1.run(a)) # generates 'A1'
    print(sess1.run(a)) # generates 'A2'
    print(sess1.run(b)) # generates 'B1'
    print(sess1.run(b)) # generates 'B2'
print("Session 2")
with tf.compat.v1.Session() as sess2:
    print(sess2.run(a)) # generates 'A3'
    print(sess2.run(a)) # generates 'A4'
    print(sess2.run(b)) # generates 'B3'
    print(sess2.run(b)) # generates 'B4'
```

```
a = tf.random.uniform([1], seed=1)
b = tf.random.normal([1])
# Repeatedly running this block with the same graph will generate
the same
# sequence of values for 'a', but different sequences of values for
'b'.
print("Session 1")
with tf.compat.v1.Session() as sess1:
    print(sess1.run(a)) # generates 'A1'
    print(sess1.run(a)) # generates 'A2'
    print(sess1.run(b)) # generates 'B1'
    print(sess1.run(b)) # generates 'B2'
print("Session 2")
with tf.compat.v1.Session() as sess2:
    print(sess2.run(a)) # generates 'A1'
    print(sess2.run(a)) # generates 'A2'
    print(sess2.run(b)) # generates 'B3'
    print(sess2.run(b)) # generates 'B4'
```

```
a = tf.random.uniform([1], seed=1)
b = tf.random.normal([1])
# Repeatedly running this block with the same graph will generate
the same
# sequence of values for 'a', but different sequences of values for
'b'.
print("Session 1")
with tf.compat.v1.Session() as sess1:
    print(sess1.run(a)) # generates 'A1'
    print(sess1.run(a)) # generates 'A2'
    print(sess1.run(b)) # generates 'B1'
    print(sess1.run(b)) # generates 'B2'
print("Session 2")
with tf.compat.v1.Session() as sess2:
    print(sess2.run(a)) # generates 'A1'
    print(sess2.run(a)) # generates 'A2'
    print(sess2.run(b)) # generates 'B3'
    print(sess2.run(b)) # generates 'B4'
```

```
tf.compat.v1.random.set_random_seed(1234)
a = tf.random.uniform([1])
b = tf.random.normal([1])
# Repeatedly running this block with the same graph will generate
the same
# sequences of 'a' and 'b'.
print("Session 1")
with tf.compat.v1.Session() as sess1:
    print(sess1.run(a)) # generates 'A1'
    print(sess1.run(a)) # generates 'A2'
    print(sess1.run(b)) # generates 'B1'
    print(sess1.run(b)) # generates 'B2'
print("Session 2")
with tf.compat.v1.Session() as sess2:
    print(sess2.run(a)) # generates 'A1'
    print(sess2.run(a)) # generates 'A2'
    print(sess2.run(b)) # generates 'B1'
    print(sess2.run(b)) # generates 'B2'
```

```
tf.compat.v1.random.set_random_seed(1234)
a = tf.random.uniform([1])
b = tf.random.normal([1])
# Repeatedly running this block with the same graph will generate
the same
# sequences of 'a' and 'b'.
print("Session 1")
with tf.compat.v1.Session() as sess1:
    print(sess1.run(a)) # generates 'A1'
    print(sess1.run(a)) # generates 'A2'
    print(sess1.run(b)) # generates 'B1'
    print(sess1.run(b)) # generates 'B2'
print("Session 2")
with tf.compat.v1.Session() as sess2:
    print(sess2.run(a)) # generates 'A1'
    print(sess2.run(a)) # generates 'A2'
    print(sess2.run(b)) # generates 'B1'
    print(sess2.run(b)) # generates 'B2'
```

Documentation

Sets the graph-level random seed for the default graph.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.random.set_random_seed`

Migrate to TF2

'`tf.compat.v1.set_random_seed`' is compatible with eager mode. However, in eager mode this API will set the global seed instead of the graph-level seed of the default graph. In TF2 this API is changed to `tf.random.set_seed`.

Description

Used in the notebooks

- TensorFlow Constrained Optimization Example Using CelebA Dataset
- Wiki Talk Comments Toxicity Prediction
- Fitting Dirichlet Process Mixture Model Using Preconditioned Stochastic Gradient Langevin Dynamics

Operations that rely on a random seed actually derive it from two seeds:

the graph-level and operation-level seeds. This sets the graph-level seed.

Its interactions with operation-level seeds is as follows:

- If neither the graph-level nor the operation seed is set:

A random seed is used for this op.

- If the graph-level seed is set, but the operation seed is not:

The system deterministically picks an operation seed in conjunction with the graph-level seed so that it gets a unique random sequence. Within the same version of tensorflow and user code, this sequence is deterministic. However across different versions, this sequence might change. If the code depends on particular seeds to work, specify both graph-level and operation-level seeds explicitly.

- If the graph-level seed is not set, but the operation seed is set:

A default graph-level seed and the specified operation seed are used to determine the random sequence.

- If both the graph-level and the operation seed are set:

Both seeds are used in conjunction to determine the random sequence.

To illustrate the user-visible effects, consider these examples:

To generate different sequences across sessions, set neither graph-level nor op-level seeds:

To generate the same repeatable sequence for an op across sessions, set the seed for the op:

To make the random sequences generated by all ops be repeatable across sessions, set a graph-level seed: